

IoT Based Smart Parking Management System

Project Team

Ifra Batool	21I-0543
Ayesha Saeed	21I-2457
Aiman Karim	21I-0664

Session 2021-2025

Supervised by

Dr. Adnan Tariq

Co-Supervised by

N/A



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.4	Modules	2
1.4.1	Module 1	2
1.4.2	Module 2	3
1.4.3	Module 3	3
1.4.4	Module 4	3
1.5	User Classes and Characteristics	4
2	Project Requirements	7
2.1	Use-case/Event Response Table/Storyboarding	7
2.2	Functional Requirements	8
2.2.1	Module 1: Sensor Data Handling	8
2.2.2	Module 2: User Interface and Real time Monitoring	8
2.2.3	Module 3: Reservation and Billing	9
2.3	Non-Functional Requirements	9
2.3.1	Reliability	9
2.3.2	Usability	9
2.3.3	Performance	9
2.3.4	Security	10
2.4	Domain Model	10
3	System Overview	13
3.1	Architectural Design	13
3.2	Design Models	15
3.3	Data Design	18
3.3.1	Information Domain to Data Structures	18
3.3.2	Major Data/System Entities	19
3.3.3	Databases or Data Storage Items	20
3.3.4	Flow of Information in the System	21

References

23

List of Figures

List of Tables

Chapter 1

Introduction

The IoT Based Smart Parking Management System aims to enhance the user experience by providing a seamless car parking solution through the use of IoT technology. The purpose of this document is to give a brief overview of what the system wants to achieve by implementing the proposed solution. The system focuses on utilizing existing parking spaces and maximizes their use by introducing a mobile application that will help monitor available parking spots, make reservations, and pay for parking. This will eliminate the hassle of long queues, unattended vehicles, difficulty in finding available spaces, and traditional payment methods. [1].

1.1 Problem Statement

So, you approach the mall so you can get some shopping done and you see that the road is blocked with traffic. You spend your valuable time circling around the parking lot hoping to find an empty parking space. After continuous searching, you ask the parking attendant if there is an available spot and he vaguely points you in a direction. You drive there only to find out that the parking space is already taken. After wasting 30 minutes to an hour of searching time you finally find a spot where you park your car and go get your shopping done. [2].

Now, as you return to your car, you are ready to pay for the parking and head home, you realize that the parking attendant is nowhere to be found. You waste even more time looking for him, adding to your frustration.

To solve this problem, we have introduced an IoT Based Smart Parking Management System. This innovative system leveraged the power of IoT technology to create a seamless parking experience. [3].

1.2 Scope

The scope of this project involves creating an IoT Based Smart Parking Management System with several key deliverables. First, we will install and configure **IoT sensors** in specific locations within our model city to monitor parking spaces and provide real time updates. We will also develop a **mobile application** where users can view available parking slots, make reservations, and handle payments all in one place. Alongside this, **database integration** with firebase will ensure that the data from the sensors is accurately stored and updated, allowing for smooth operation. To ensure the system works as expected, we will conduct a pilot deployment in our model city, which will serve as a testing ground before considering broader use. Additionally, **user documentation** will be created to guide both the users of the mobile application and those managing the IoT infrastructure.

The **project boundaries** include an initial deployment in major urban areas of Pakistan, targeting high demand locations like commercial zones, public facilities, and residential complexes. The system will utilize **existing IoT technologies** without developing custom hardware. During the **pilot phase**, the system will manage up to 1,000 parking slots, scaling up as needed based on demand and capacity. The **target audience** includes urban commuters, residents of large complexes, and visitors to public and commercial areas, ensuring a broad user base. [4].

1.3 Modules

Write down the modules of the proposed project. Each module should highlight features, using bulleted/numbered notation. When developing both a mobile app and a web app, group the modules according to the system types, such as, Client Web App, Client Mobile App, Admin Web App etc.

1.4 Modules

1.4.1 Module 1

This module deals with sensor selection. Of the many sensors available, it is of utmost importance that a suitable sensor is selected. These sensors will calculate the distance between them and the car, to determine if the slot is vacant or occupied. The sensors selected will be readily available in the market.

1. Select a suitable sensor and microcontroller to pair
2. Reading data from sensors
3. Sending data from sensors to the cloud

1.4.2 Module 2

This module focuses on the mobile app development and monitoring of slots, the Figma design, the app development, and the real time monitoring of the vacant and/or available parking slot. The app will provide a smooth ride to the system, allowing users to use the app interactively, where they can see, in real-time, the vacant slots so they may choose a slot to their desire.

1. System design, Figma prototyping, and initial development
2. Real time Monitoring

1.4.3 Module 3

This module focuses on the reservation as well as billing features. Another major component of the system is the ability to reserve a slot. The user may be able to see the vacant slot and be able to reserve the slot they desire. Additionally, the system also has a seamless billing option.

1. Reservation of slots
2. Seamless billing

1.4.4 Module 4

The final module deals with any changes or improvements in the system. Should something go wrong or should something take longer than expected, this module allows for quick and efficient changes.

1. Improve UI/UX
2. Other improvements

1.5 User Classes and Characteristics

Identify the various user classes that you anticipate will use this product, and describe their pertinent characteristics.

User class	Description
Car Owners	Car owners are our primary users. They will use our mobile application to check the availability of parking spaces, make reservations, and pay for parking online. And hence they are expected to use the system frequently, especially during busy hours or in high demand areas. Therefore, they need a user friendly interface that provides real time updates on parking availability and payment options.
Parking Lot Administrators	Admin users are responsible for managing the parking spaces. They will use a back end system to monitor the occupancy of the parking lot, manage reservations, and ensure that all payments are processed smoothly. They require access to real time data and analytics to maintain efficiency.
Payment Handlers	This includes banks or payment gateways, will facilitate the digital payment transactions within the system. They will ensure that payments made through the app are processed securely and efficiently. Payment handlers need integration with secure payment protocols and compliance with financial regulations.
Government Authorities	These people will will oversee the implementation and operation of the parking system. Also ensure compliance with public regulations. They need access to transactional records and data reports to monitor operations for tax purposes.

User Classes and Characteristics

Chapter 2

Project Requirements

2.1 Use-case/Event Response Table/Storyboarding

For the IoT based Smart Parking Management System, we will use an Event Response Table, as our system primarily performs real time functions in the back end for instance the sensor data monitoring for slot availability and reservation management. This representation will outline the key events and their respective system responses as given below.

Event	System Response
User checks parking availability	The app fetches real time slot availability from the Firebase detected by the sensors.
A car enters a parking space	The sensor detects the car, marks the spot as "occupied" in the database.
A car leaves the parking space	The sensor detects the car leaving, marks the spot as "available" in the database.
User reserves a parking slot	The system updates the database to mark the spot as "reserved"
User completes the parking session	The system processes payment and updates the spot as "available"
Sensor fails to detect a vehicle	The system sends an alert for sensor administrators

Event Response Table

2.2 Functional Requirements

2.2.1 Module 1: Sensor Data Handling

The first module deals with the selection of suitable sensors and reading data from them.

1. The system shall read data from an ultrasonic sensors to determine whether a parking slot is occupied or available.
2. The system shall transmit sensor data to the Firebase for storage and real time updates on the app.

2.2.2 Module 2: User Interface and Real time Monitoring

This module is focused on the development of the user facing mobile application and its monitoring features.

1. The mobile app shall display real time availability of parking slots based on sensor data.

2. The mobile app shall allow users to reserve a parking spot through an interactive interface.
3. The system shall notify users when a reserved parking spot is about to expire.

2.2.3 Module 3: Reservation and Billing

This module handles reservation and payment functionalities.

1. The system shall allow users to reserve parking spots in advance.
2. The system shall provide seamless payment options for parking sessions.

2.3 Non-Functional Requirements

2.3.1 Reliability

1. Talking about MTBF, we will ensure that we achieve a MTBF of 99.5 percentage uptime this will ensure the system only fails a maximum of 2.5 days per year.
2. The system will keep and install backup sensor systems in case of failure, ensuring continuous operation.

2.3.2 Usability

The app must be intuitive and require no more than 5 minutes for a new user to understand all the main features.

1. The system shall allow a user to reserve a parking spot in 3 clicks or less.
2. The system will generate an error message when a users attempt to reserve an already occupied spot, alternatively suggesting an available spots.

2.3.3 Performance

1. The system shall update the parking slot availability within 5 seconds of a car entering or leaving the parking lot.
2. The mobile app shall load parking availability information within 3 seconds when accessed by users.

3. In later stages, the database shall handle up to 1,000 parking spots in the initial phase without performance degradation.

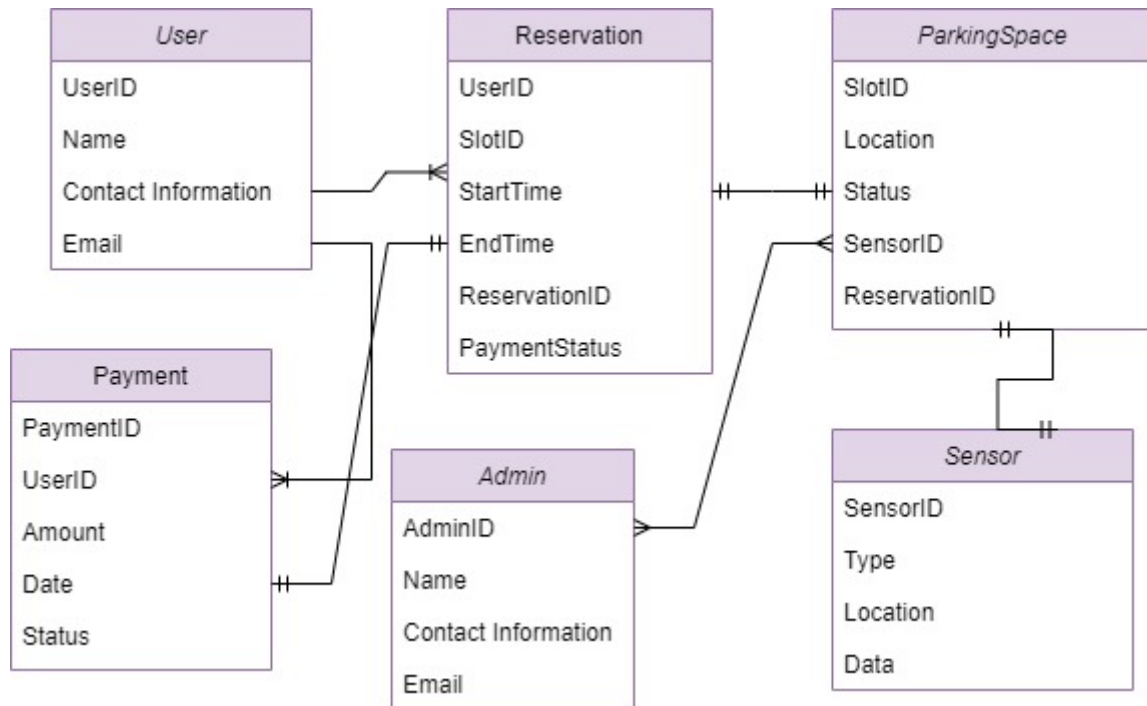
2.3.4 Security

1. The system shall enforce a minimum of 2 factor authentication for users logging into the mobile app.
2. Access to the database shall be restricted to authorized personnel only, with logs of all access attempts maintained.

2.4 Domain Model

Entities: User, Admin, Reservation, Parking Space, Payment, and Sensor. User: UserID, Name, Contact Information, and Email. Users make reservations and payments. Admin: Manages the system, identified by AdminID, and has fields such as Name, Contact Information, and Email. Reservation: Tracks the parking reservation made by users, with details like ReservationID, UserID, SlotID, StartTime, EndTime, and PaymentStatus. ParkingSpace: Contains parking slot details, including SlotID, Location, Status, SensorID, and ReservationID. Payment: Stores payment-related information with PaymentID, UserID, Amount, Date, and Status. Sensor: Represents the parking sensors, containing SensorID, Type, Location, and Data.

The relationships between the entities define how users reserve slots, make payments, and how sensors track parking space availability.



Chapter 3

System Overview

The IoT Based Smart Parking Management System combines hardware sensors for vehicle detection with Firebase for data storage, accessible through a mobile app for real-time updates, reservations, and payments. We've created an ERD for the database, use case and system sequence diagrams for user interactions, a state machine diagram for parking spot status, and an activity diagram for user flow. These diagrams collectively outline the system's functionality and data processes.

3.1 Architectural Design

The IoT based smart car parking system is designed with a multi-layered architecture for separation and clear communication with each layer.

The first layer is the user interface, a mobile application with easy user interaction and navigation.

- The app allows for key functionality to be visualized by users and it communicates with backend to process the retrieved data.

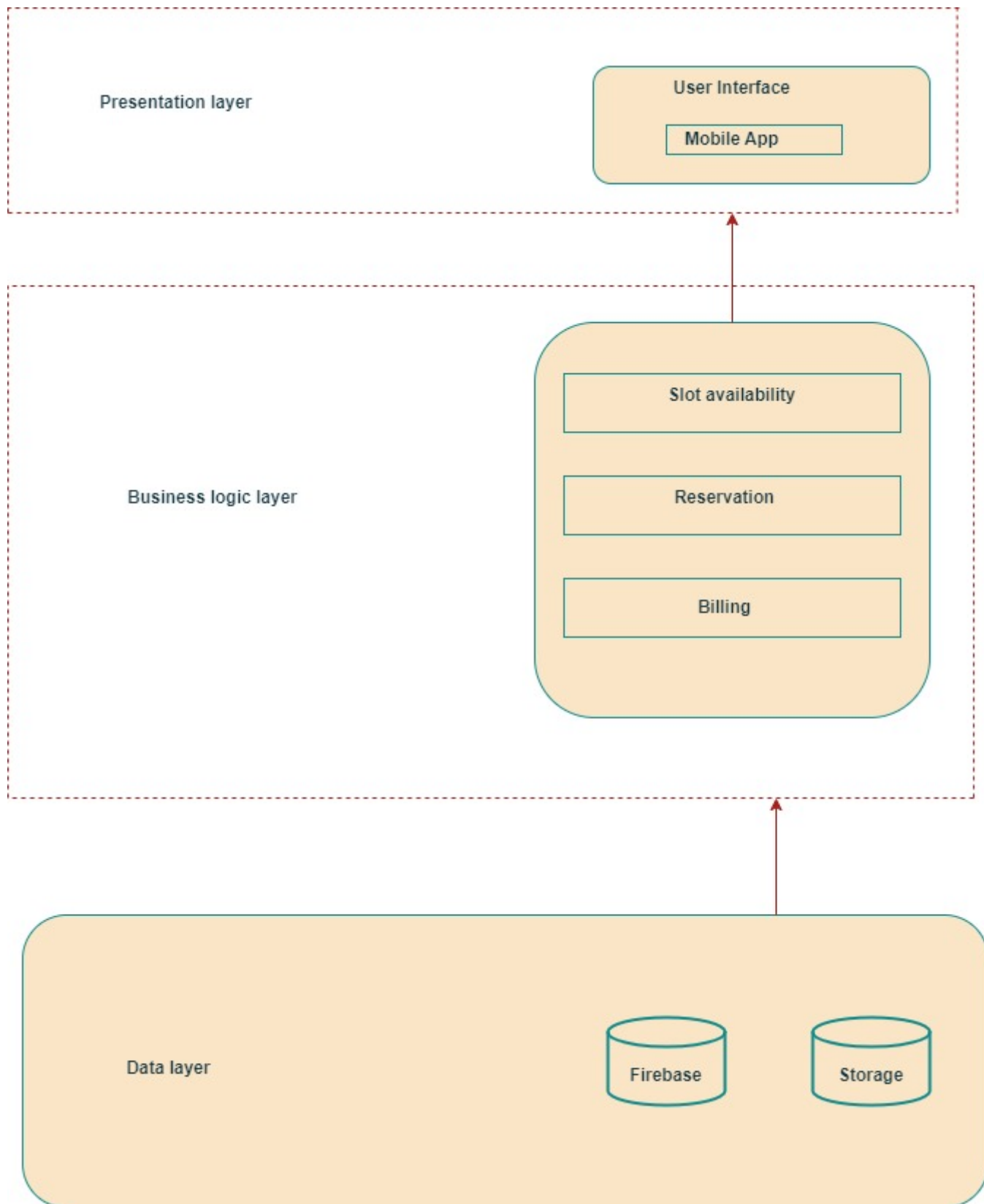
The business logic layer deals with the inner fine working of the application where the retrieved data is processed.

- It works on the reservation and monitoring functionality.
- Also works as a bridge between database layer and user interface layer.

The last layer is that of database layer where all data is stored.

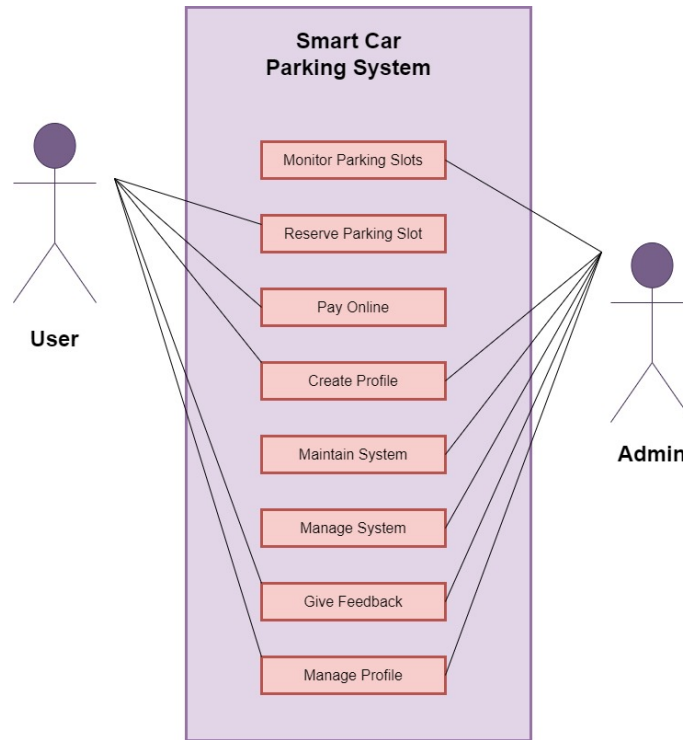
- The database holds critical information such as parking availability, reservation history, and payment records.

- It ensures that the application always has access to the most up-to-date information on parking slot statuses and reservations.

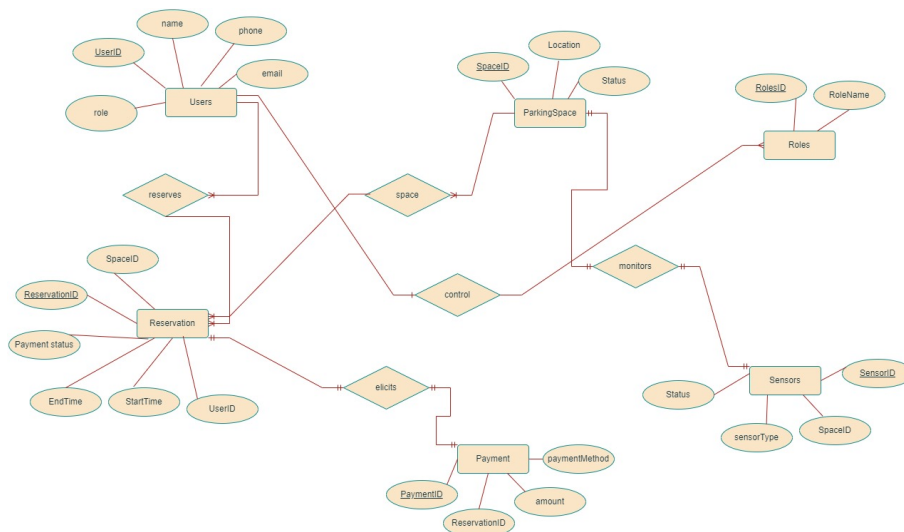


3.2 Design Models

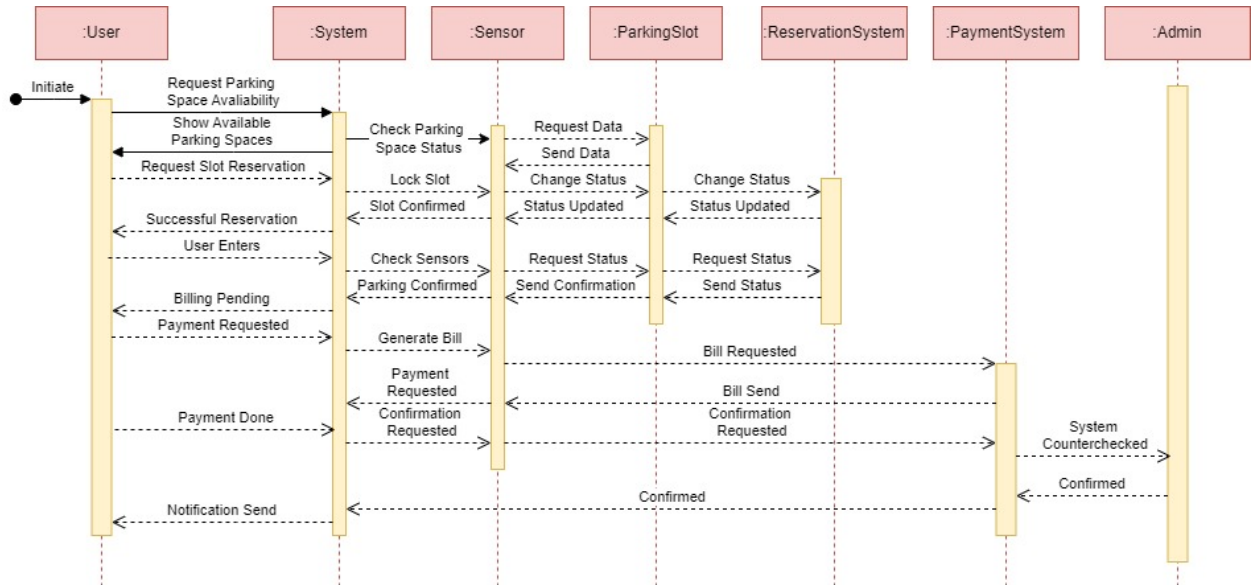
This **Use Case diagram** reflects all possible interactions that will be made between the system actors both internal and external, and the system itself. As of now our system has no external actor but System Administration and the system user. We have multiple use cases reflecting the core functionalities our system will be performing.



The **ERD** provides a visual depiction of the database structure, Highlighting the entities, the attributes, and their relationship with each other. The diagram helps to understand how each entity interacts with each other and the system. The entities are: Users, Reservation, ParkingSpace, Payments, Sensors and Roles each with their attributes.

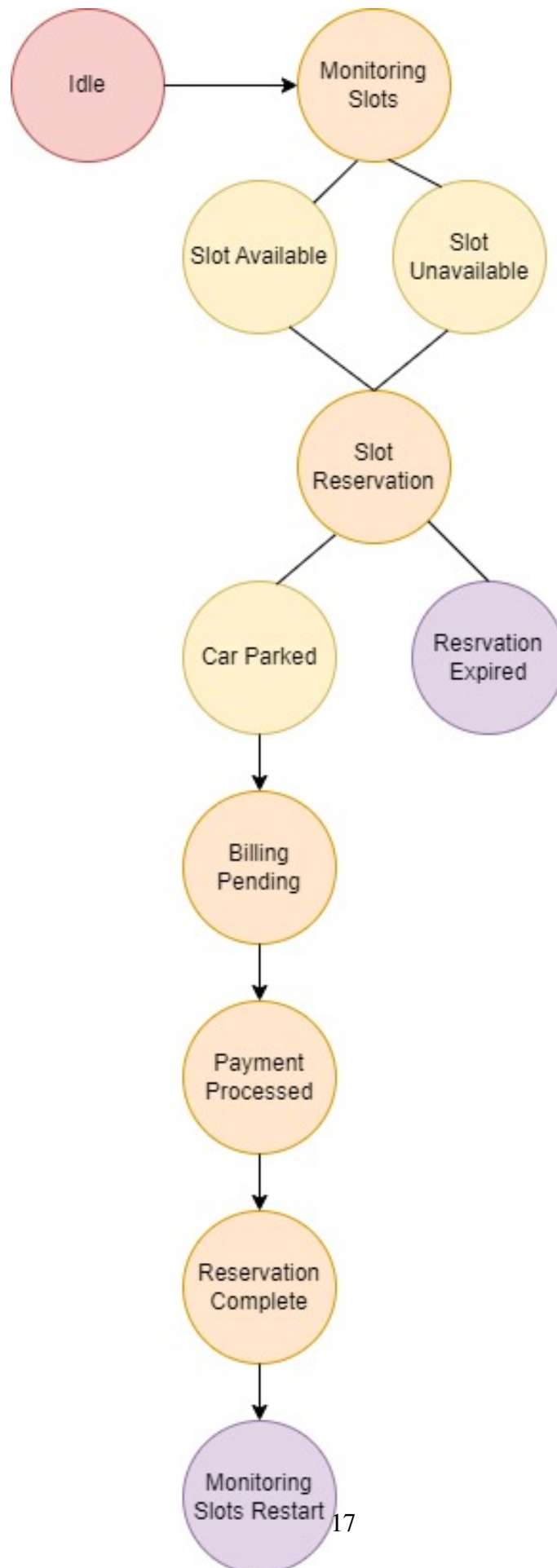


The **SSD** gives an overall explanation of how the Smart Car Parking System works and drafts system requirements. It highlights the objects in the system and their communication protocol, in our system we have 7 major objects, each of them plays a crucial role in running the system successfully. Each object lifecycle is initiated when they're called by other objects and ends when they have performed their asked function.

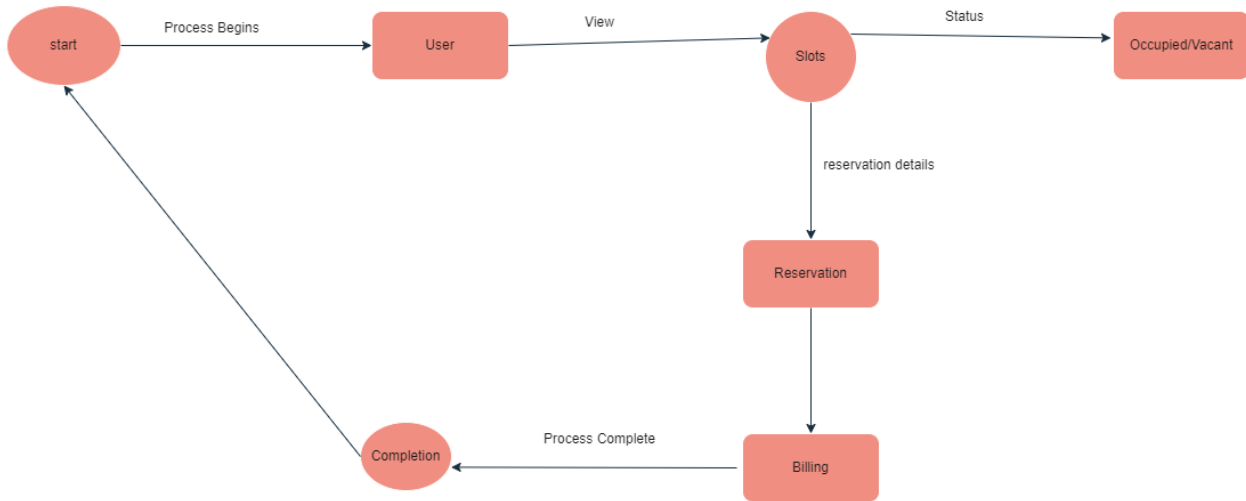


The **State Transition diagram** has the following:

- **Idle State:** The system starts in an idle state, monitoring parking slots.
- **Slot Monitoring:** When slots become available or unavailable, the system updates the status accordingly.
- **Slot Reservation:** Users can reserve available slots, with the reservation expiring if not used within the allocated time.
- **Car Parked:** Once a car is parked, the system moves to the billing.
- **Payment:** The user is asked to pay, and once payment is processed, the reservation is completed.
- **Monitoring Restarts:** After a reservation is complete, the system resumes monitoring the slots.



The **Data Flow diagram** shows the flow of data from the start of the process to the end, showing how data is processed and then transferred between various entities. It serves as a crucial tool for stakeholders to easily understand the flow and the system functionality and a clear view of data interactions.



3.3 Data Design

3.3.1 Information Domain to Data Structures

The system's information domain can be divided into several important parts or entities. These entities can be represented using different data structures, which will later be stored in a database.

- **Parking Slots:**

- **Attributes:** Slot ID, status (vacant/occupied), location (latitude, longitude), sensor ID.
- **Data Structure:** This is shown as a table where each parking slot is a record, and its status is updated each time there is a sensor input.

- **Reservations:**

- **Attributes:** Reservation ID, Slot ID, User ID, reservation timestamp, expiration timestamp.
- **Data Structure:** A table for reservations, with each record holding a reference value to the user and the parking slot reserved, along with their timestamps.

- **Users:**

- **Attributes:** User ID, name, email, vehicle registration number, current reservation (if any).
- **Data Structure:** A table where each entry is a user profile, allowing easy access to the user's reservation and payment details.
- **Payments:**
 - **Attributes:** Payment ID, User ID, amount, payment method, status (pending/complete), timestamp.
 - **Data Structure:** A table of payment transactions, linked to users and reservations.
- **Sensor Data:**
 - **Attributes:** Sensor ID, Slot ID, status (occupied/vacant), timestamp.
 - **Data Structure:** A continuous feed of sensor data, stored for tracking slot availability.

3.3.2 Major Data/System Entities

Each major entity plays a specific role in the system, and they interact with each other based on user actions and sensor inputs.

- **Parking Slots:**
 - **Stored:** In a relational database, each parking slot is a row in a `Parking_Slots` table with attributes: Slot ID, status, and sensor ID.
 - **Processed:** The status of each slot is updated based on inputs from the sensor. If a user reserves a spot, the status is temporarily locked.
 - **Organized:** Indexed by Slot ID to quickly retrieve available slots or update the slot's status.
- **Reservations:**
 - **Stored:** In a Reservations table, with references to the user and the parking slot.
 - **Processed:** When a user reserves a slot, a new reservation record is created, and the slot's status is marked as "reserved". After the reservation expires or the parking session ends, the reservation is removed or marked as completed.
 - **Organized:** By Reservation ID and User ID for efficient lookup of ongoing and past reservations.

- **Users:**
 - **Stored:** In a Users table, containing user details, current reservations, and payment information.
 - **Processed:** When a user makes a reservation or payment, their record is updated with new information.
 - **Organized:** Indexed by User ID for quick access to user details during login, reservation, or payment processing.
- **Payments:**
 - **Stored:** In a Payments table, with references to the user and reservation.
 - **Processed:** When a user completes their parking session, a payment record is generated and linked to the reservation.
 - **Organized:** By Payment ID and User ID for tracking payment history.
- **Sensor Data:**
 - **Stored:** In a time-series database or a SensorLogs table, with each entry recording the status of a slot at a given time.
 - **Processed:** The system continuously monitors this data, updating the availability of slots in real time.
 - **Organized:** Indexed by Sensor ID and Slot ID for fast retrieval of slot statuses.

3.3.3 Databases or Data Storage Items

The system would require a combination of databases to store different types of information:

- **Cloud Storage (like Firebase):** Could be used for storing the entire backend data, handling both structured data (slots, reservations, payments) and real-time data (sensor logs).
- **NoSQL Database (e.g., Firebase Firestore, MongoDB):** Ideal for storing semi structured data like user profiles and handling real time updates for parking slot statuses and reservations.

3.3.4 Flow of Information in the System

- **User Interaction:**
 - When the user opens the mobile app, the system queries the Parking_Slots table for available slots and displays them.
 - Once the user selects a slot, the Reservations table is updated, and the status of the parking slot in ParkingSlots is updated.
- **Sensor Data Processing:**
 - IoT sensors continuously send data to the SensorLogs table, which is processed by the back end to update the status of slots in real-time.
- **Payments:**
 - When a user completes a parking session, a record is created in the Payments table, linking the user and reservation information.

Bibliography

- [1] M Awais A R Mohsin, M Khalid and K Ahmad. Real-time car parking system using arduino control. *archive.conscientiabeam*, 2021.
- [2] A Khanna and R Anand. Iot based smart parking system. *International Conference on Internet of Things and Applications*, 2016.
- [3] S Koba. Iot based smart parking system development. *Mobidev.biz*, 2024.
- [4] L Terashvili. Iot-based smart parking system: How to create what do you need? *SirinSoftware*, 2024.